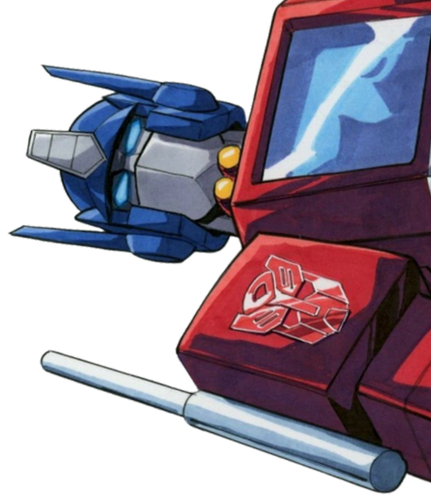


Interpretability on Transformers

VOL. 1

Sara Candussio

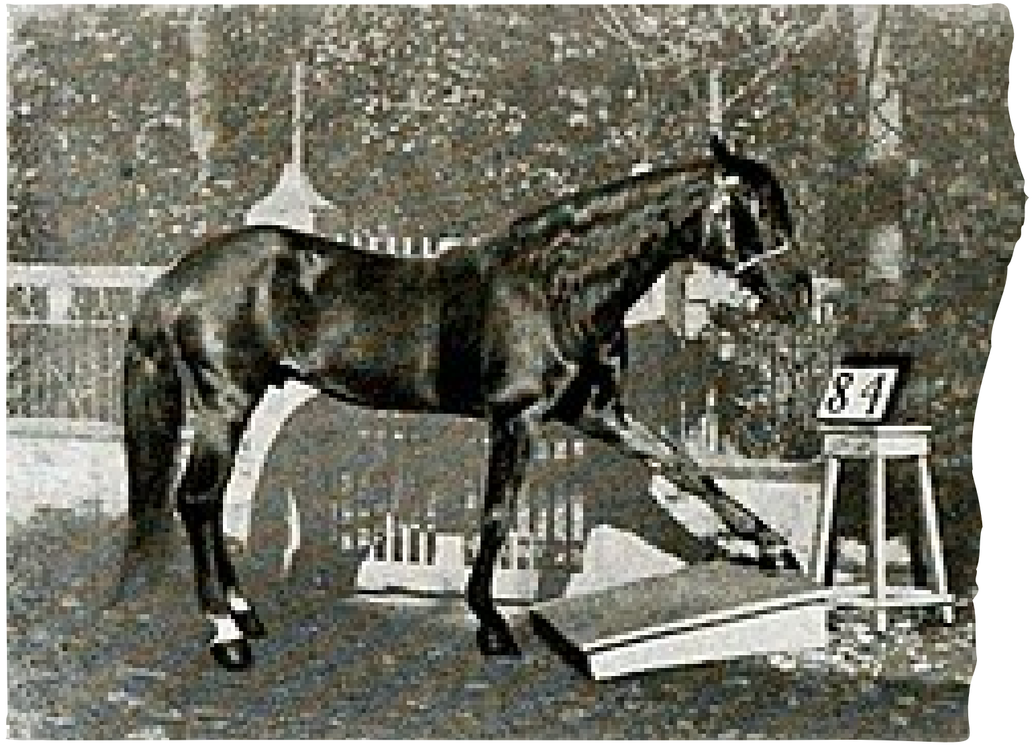
Nov 13th, 2026

Basics

what is interpretability and why do we care; the Transformer architecture

Right Predictions, Wrong Reasons

A **correct** prediction is **not sufficient** to ensure that the model complies with **user expectations**. *Models can be right for the wrong reasons*.



der Kluge Hans; c. 1894 – c. 1916

1. the **elements** and **process** leading to the prediction are **fundamental** for evaluating its correctness
2. an explaination should be evaluated in terms of **faithfulness to the model** and **plausibility to the user**.

🎯 **understand the behavior** AI systems with **human-intelligible explanations** that are **faithful to AI internal processes**

🎯 **empower** AI users with **actionable insights** from AI **predictions**.

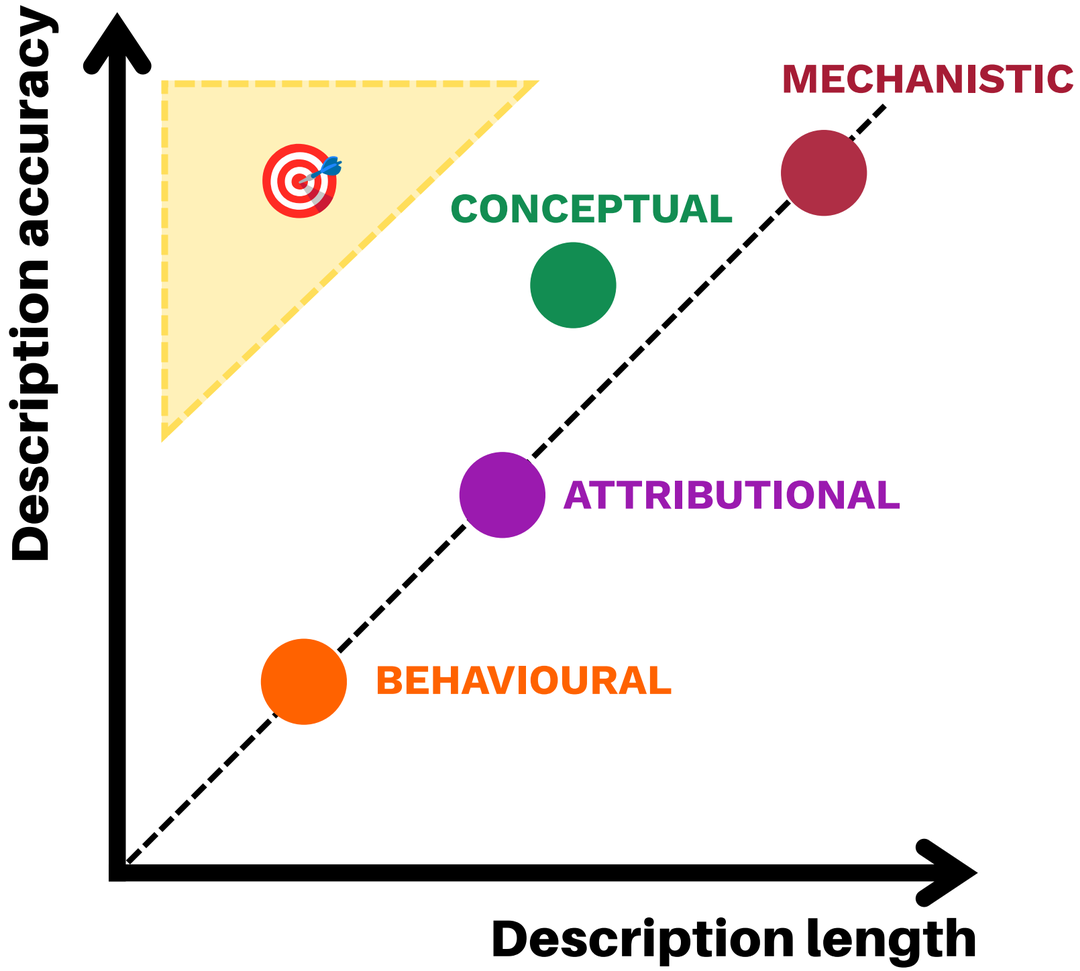
Granularity levels of interpretability

BEHAVIOURAL
How does the model **behave** in **specific settings/phenomena**?

ATTRIBUTIONAL
Which **input/training elements** are **responsible** for model predictions?

CONCEPTUAL
Which **high-level features** are **encoded** by the model?

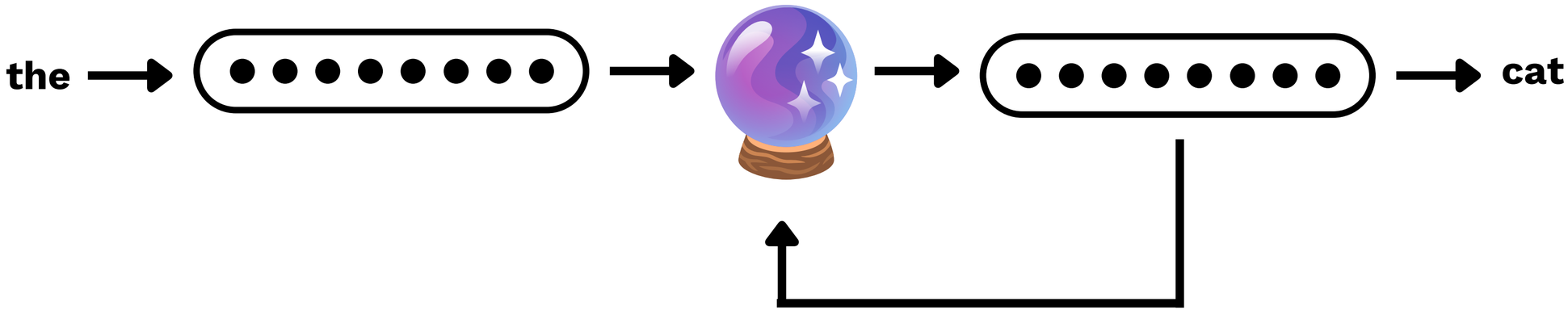
MECHANISTIC
Which **model units** are responsible for a specific behavior?

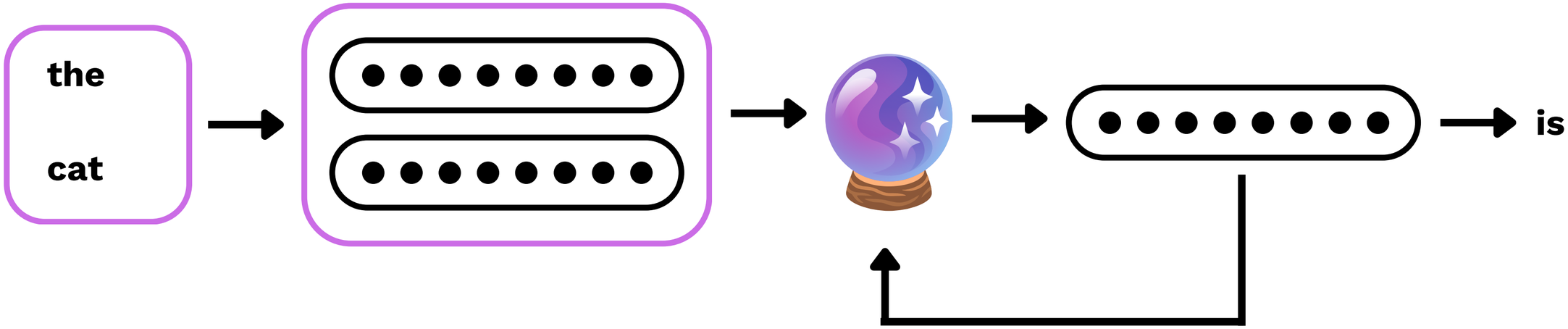


The “**holy grail**”: *precise* and *actionable* explanations that can be easily conveyed and understood.

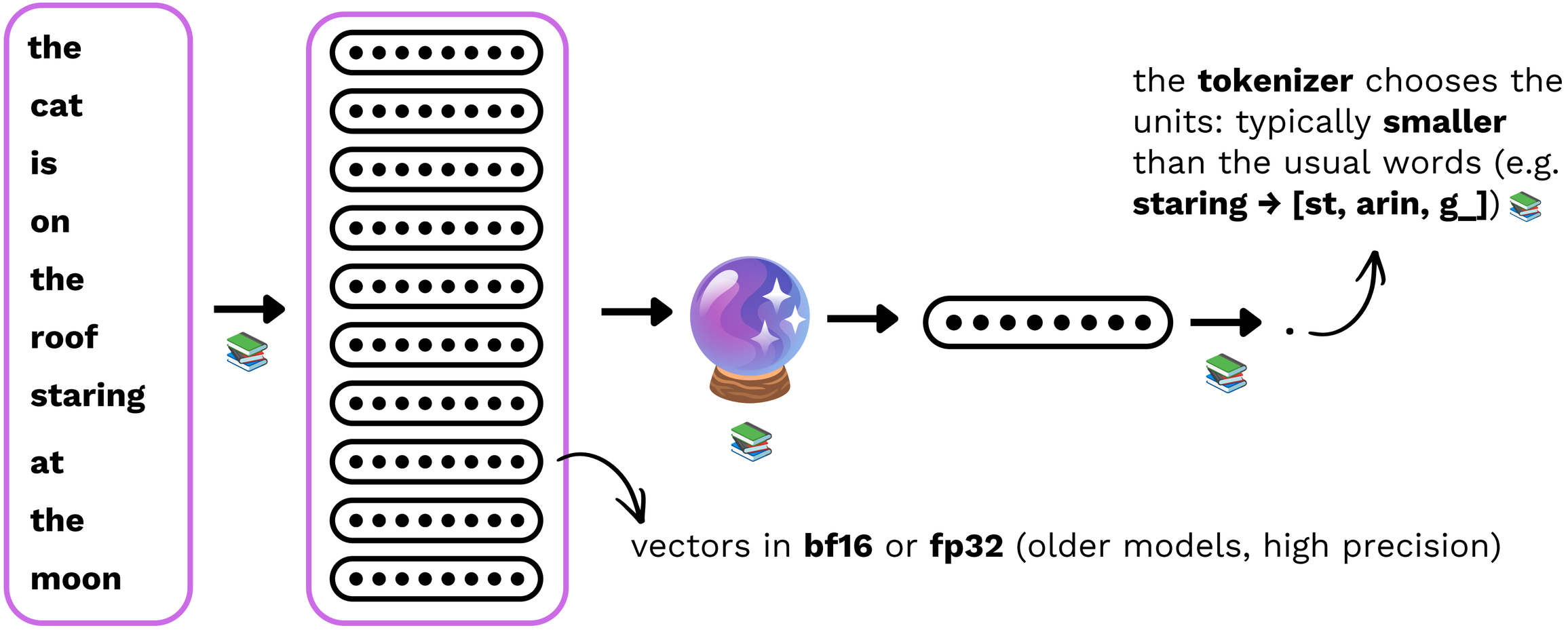
What is a Transformer?

What is a Transformer?

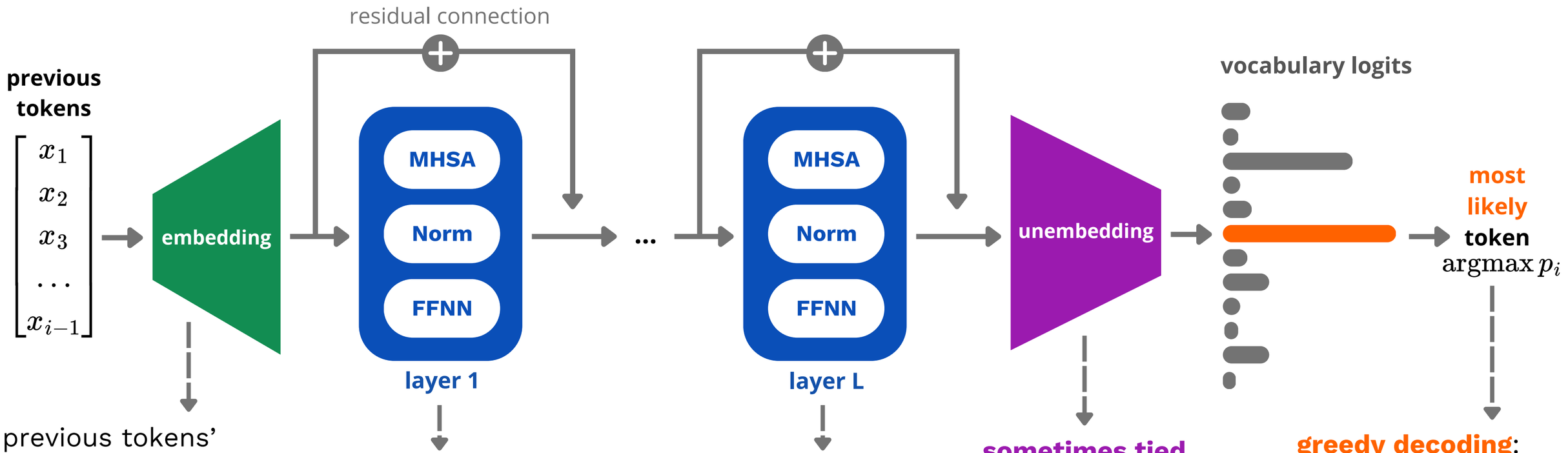




What is a Transformer?



What is a (decoder-only) Transformer?



inference = a **forward pass** in this computational graph

different **architectures** (i.e. number of layers, position of operations, ...) are present

no explicit encoder, **coupled representation + processing**

Looking inside a layer

...

Multi-Head Self-Attention (MHSA)

acts as a **focal mechanism**: each **head** decides where to look in the previous tokens (imagine it as a particular **viewpoint**) and **what to copy** from them **to the residual stream**

LayerNorm (LN)

rescales each **token's vector** to stabilize training; placed **before** (pre-) **or after** (post-) each sub-block, **architecture-dependent**

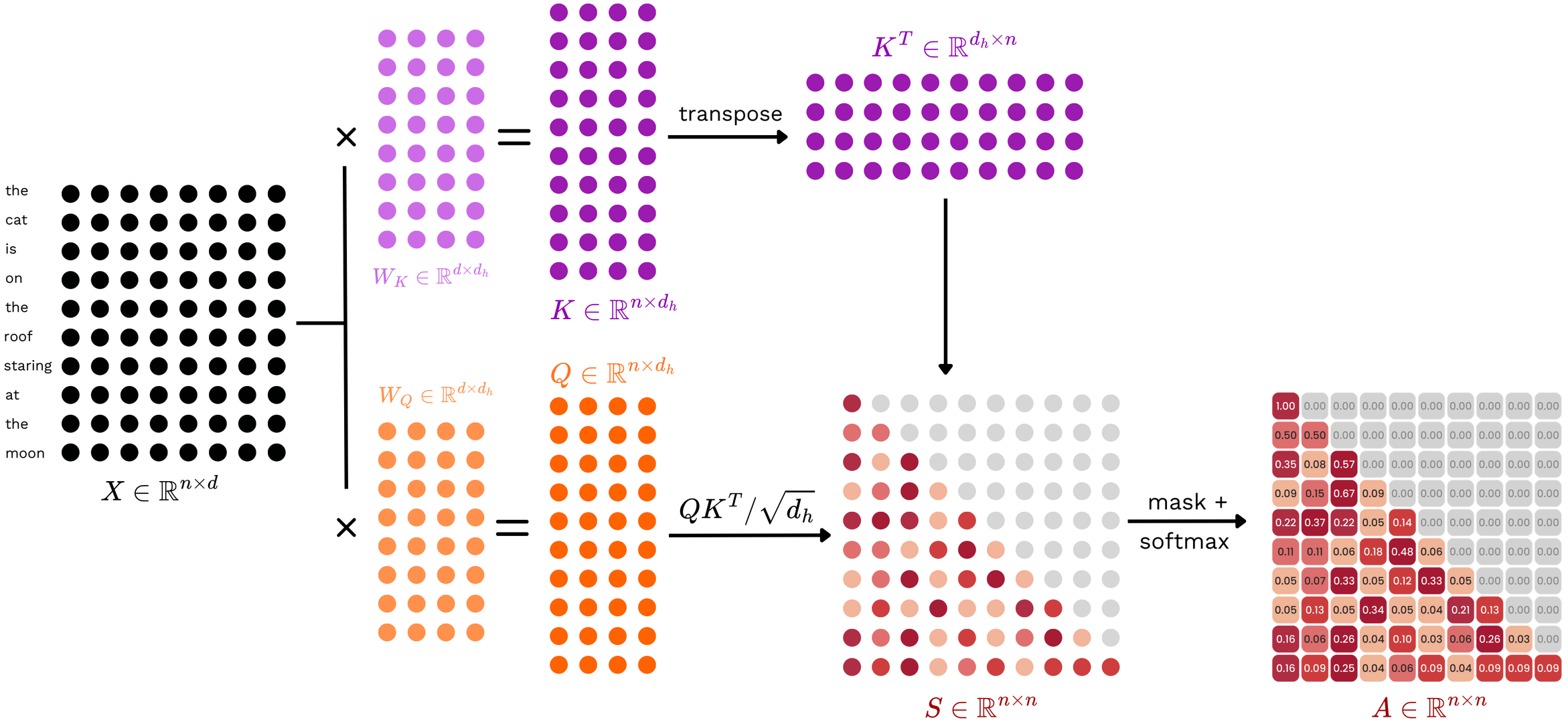
Feed-Forward Neural Network (FFNN or MLP)

a **per-position** Multi-Layer Perceptron: expands → non-linearity → projects back; often read as **a key-value memory**: *"if you see this* (key) *then write that* (value)" (Geva et al., 2021)

...

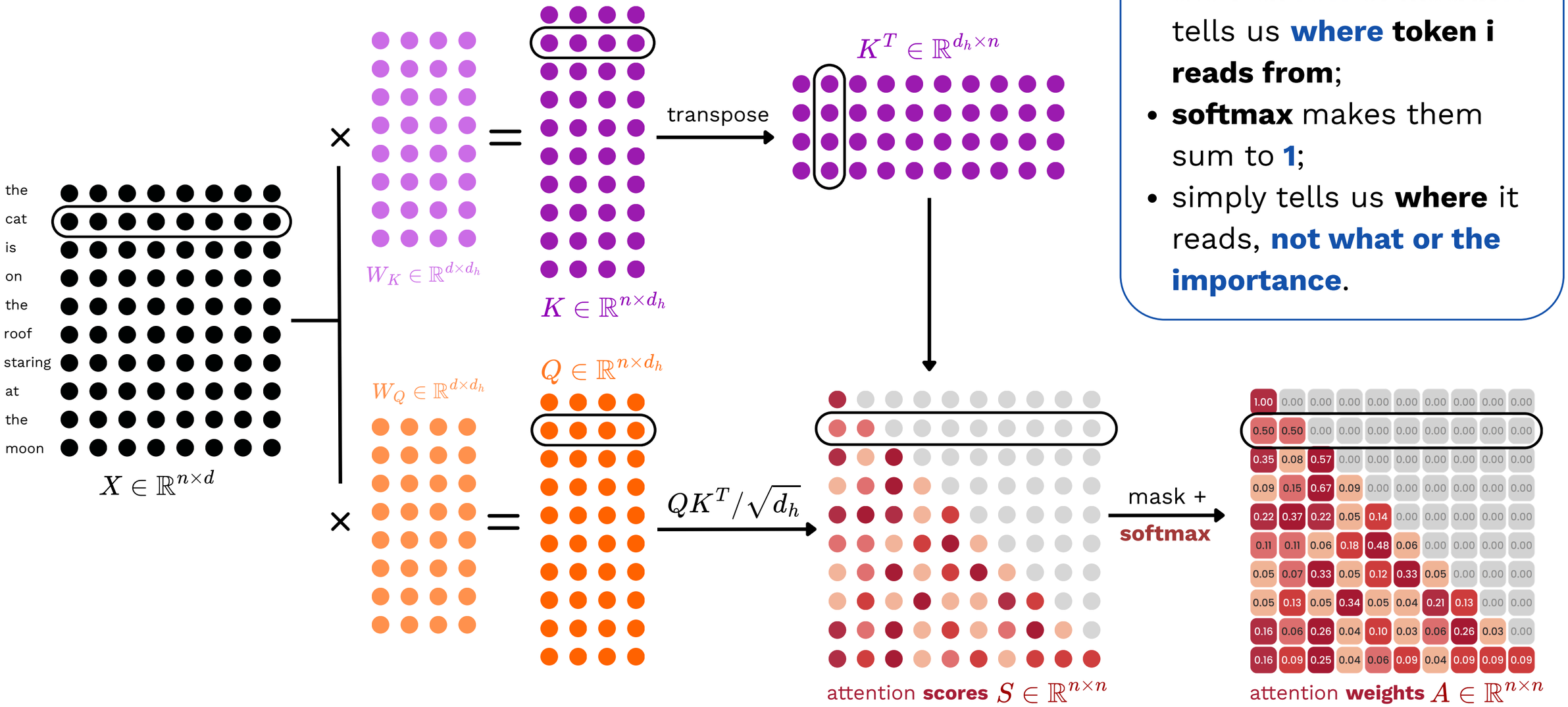
Self-Attention: where to look

Every **row** of X is projected **twice** using **query** and **key matrices:**



Self-Attention: where to look

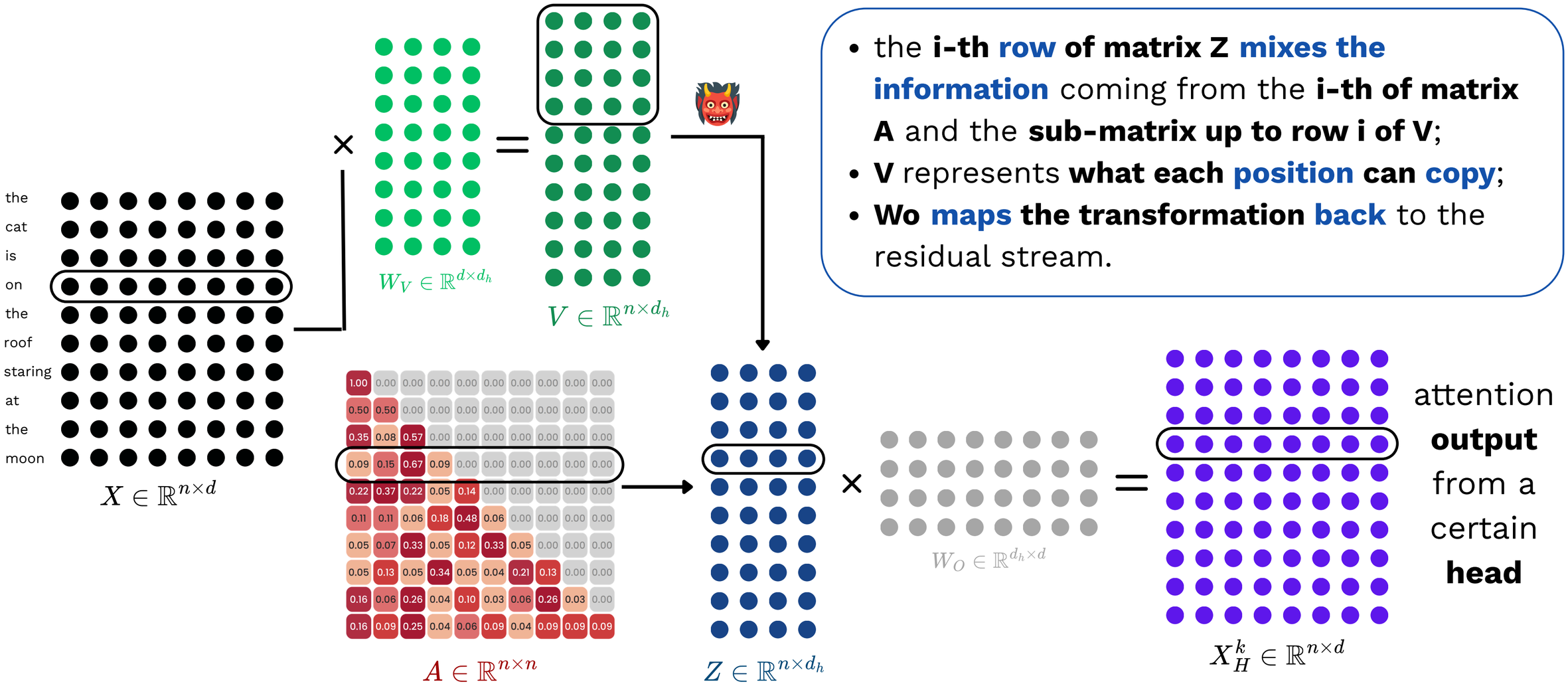
Follow what happens to the “**cat**” vector representation:



- the **i-th row of matrix A** tells us **where token i reads from**;
- **softmax** makes them sum to **1**;
- simply tells us **where** it reads, **not what or the importance**.

Self-Attention: what to write

Another matrix **V** is responsible for **choosing what to copy**

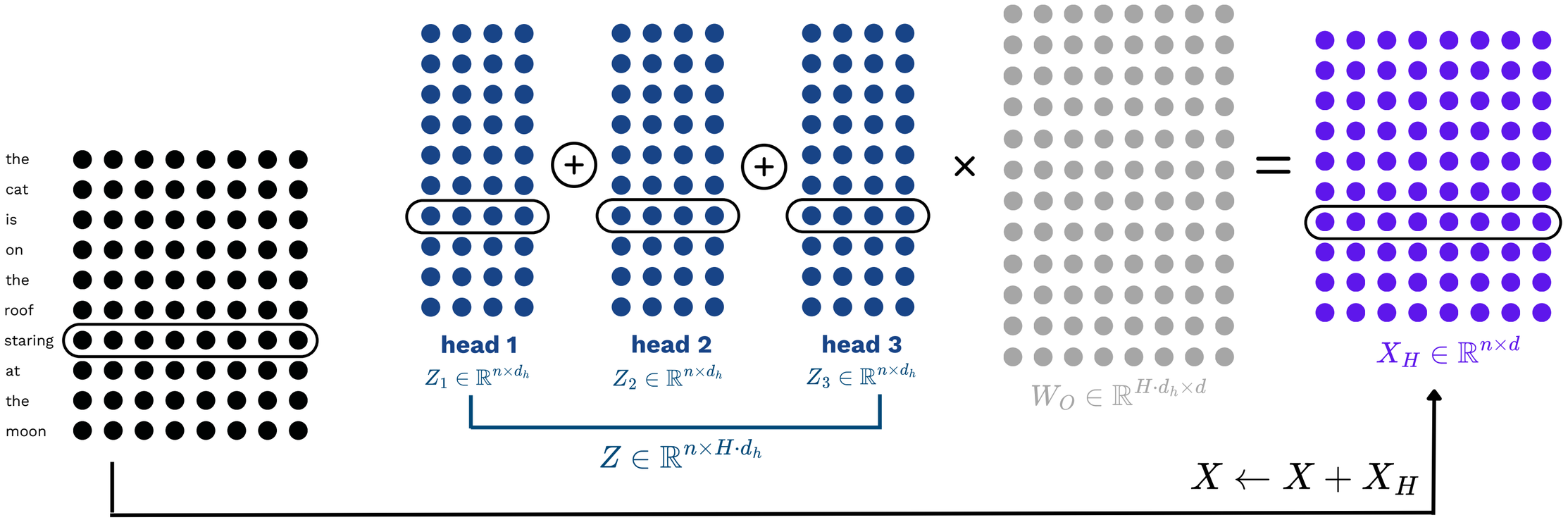


- the **i-th row of matrix Z mixes the information** coming from the **i-th of matrix A** and the **sub-matrix up to row i of V**;
- **V** represents **what each position can copy**;
- **Wo maps the transformation back** to the residual stream.

attention **output** from a certain **head**

Multi-Head Self-Attention

Why? Because in this way we can **highlight different properties** of the context.



new knowledge is merged with the **initial representation, why?**

Why do we apply **softmax to QK^T before** multiplying by **V**?

Do different **heads specialise without explicit training bias**?

Why do we need **Wq**, **Wk**, **Wv** and **Wo**?

Looking inside a layer

...

Multi-Head Self-Attention (MHSA)

acts as a **focal mechanism**: each **head** decides where to look in the previous tokens (imagine it as a particular **viewpoint**) and **what to copy** from them **to the residual stream**

LayerNorm (LN)

rescales each **token's vector** to stabilize training; placed **before** (pre-) **or after** (post-) each sub-block, **architecture-dependent**

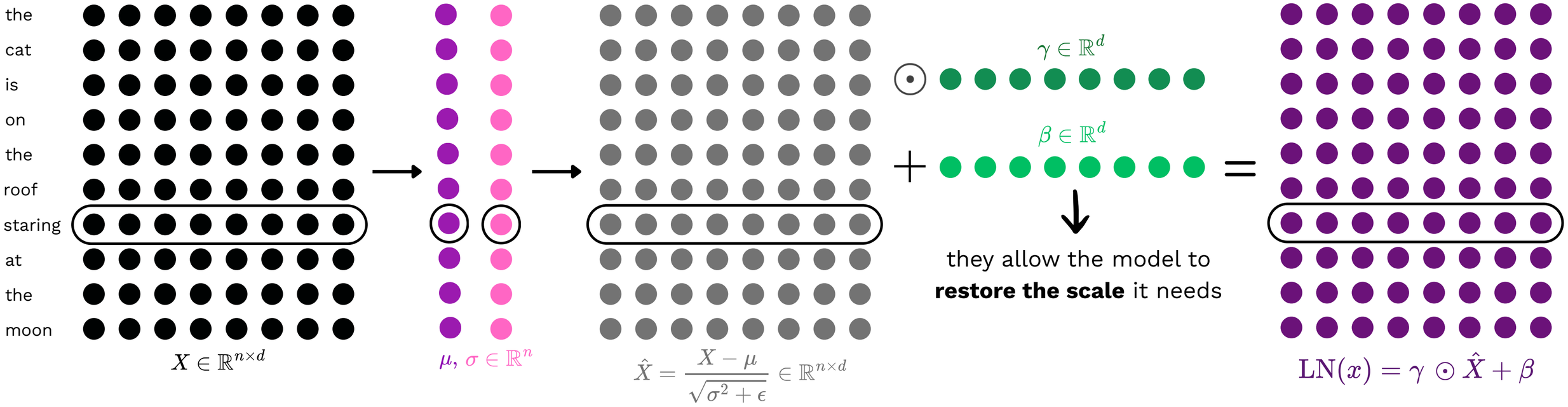
Feed-Forward Neural Network (FFNN or MLP)

a **per-position** Multi-Layer Perceptron: expands → non-linearity → projects back; often read as **a key-value memory**: *"if you see this* (key) *then write that* (value)" (Geva et al., 2021)

...

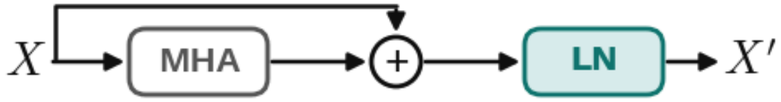
Layer Normalisation

row-normalization, i.e. every **item representation** is normalised with respect to its **features**

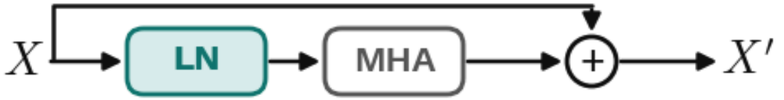


cumulative residual stream → its **norm grows with depth** → need to **normalise it**

post-LN: normalizes the stream



pre-LN: leaves the stream untouched



each block reads the input at a **stable scale** (and *stable gradients imply faster training*)

Looking inside a layer

...

Multi-Head Self-Attention (MHSA) ✓

acts as a **focal mechanism**: each **head** decides where to look in the previous tokens (imagine it as a particular **viewpoint**) and **what to copy** from them **to the residual stream**

LayerNorm (LN) ✓

rescales each **token's vector** to stabilize training; placed **before** (pre-) **or after** (post-) each sub-block, **architecture-dependent**

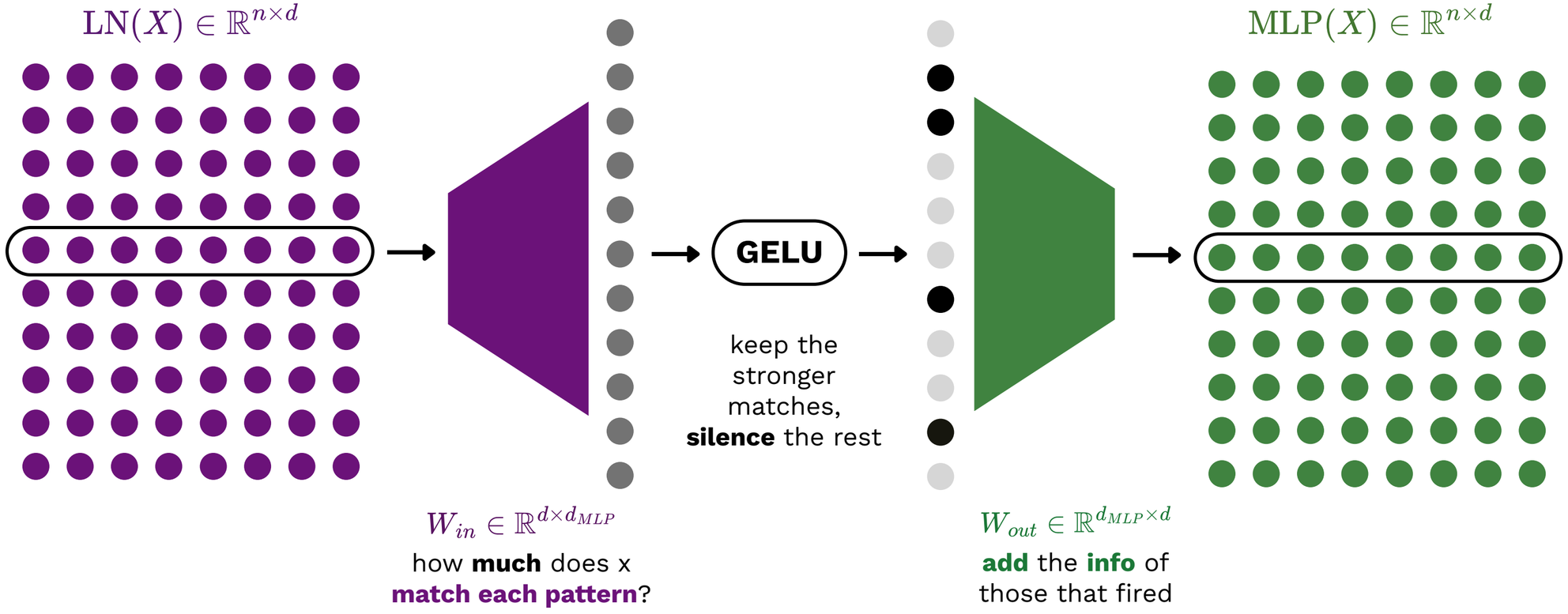
Feed-Forward Neural Network (FFNN or MLP)

a **per-position** Multi-Layer Perceptron: expands → non-linearity → projects back; often read as **a key-value memory**: *“if you see this* (key) *then write that* (value)” (Geva et al., 2021)

...

MLP: the model's memory

Each token is checked against **learned patterns** and **could add some information** to it



the two matrices are the same for every prompt → **long-term memory**

2/3 of the parameters of a Transformer are MLP **weights**

only **few** weights **per input** should fire

Looking inside a layer

...

Multi-Head Self-Attention (MHSA)

acts as a **focal mechanism**: each **head** decides where to look in the previous tokens (imagine it as a particular **viewpoint**) and **what to copy** from them **to the residual stream**

LayerNorm (LN)

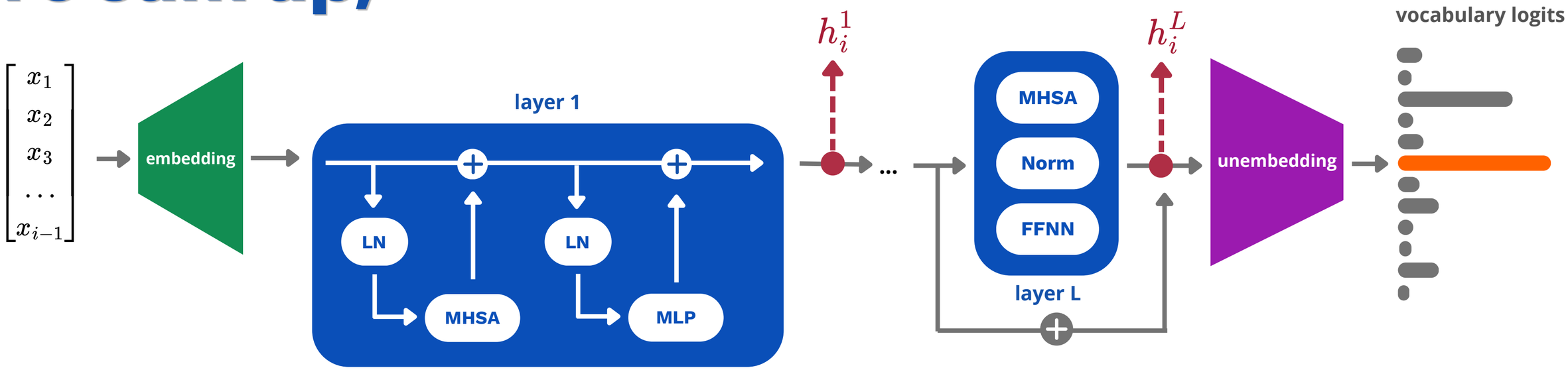
rescales each **token's vector** to stabilize training; placed **before** (pre-) **or after** (post-) each sub-block, **architecture-dependent**

Feed-Forward Neural Network (FFNN or MLP)

a **per-position** Multi-Layer Perceptron: expands → non-linearity → projects back; often read as **a key-value memory**: *“if you see this* (key) *then write that* (value)” (Geva et al., 2021)

...

To sum up,



deliberate simplifications/omissions:

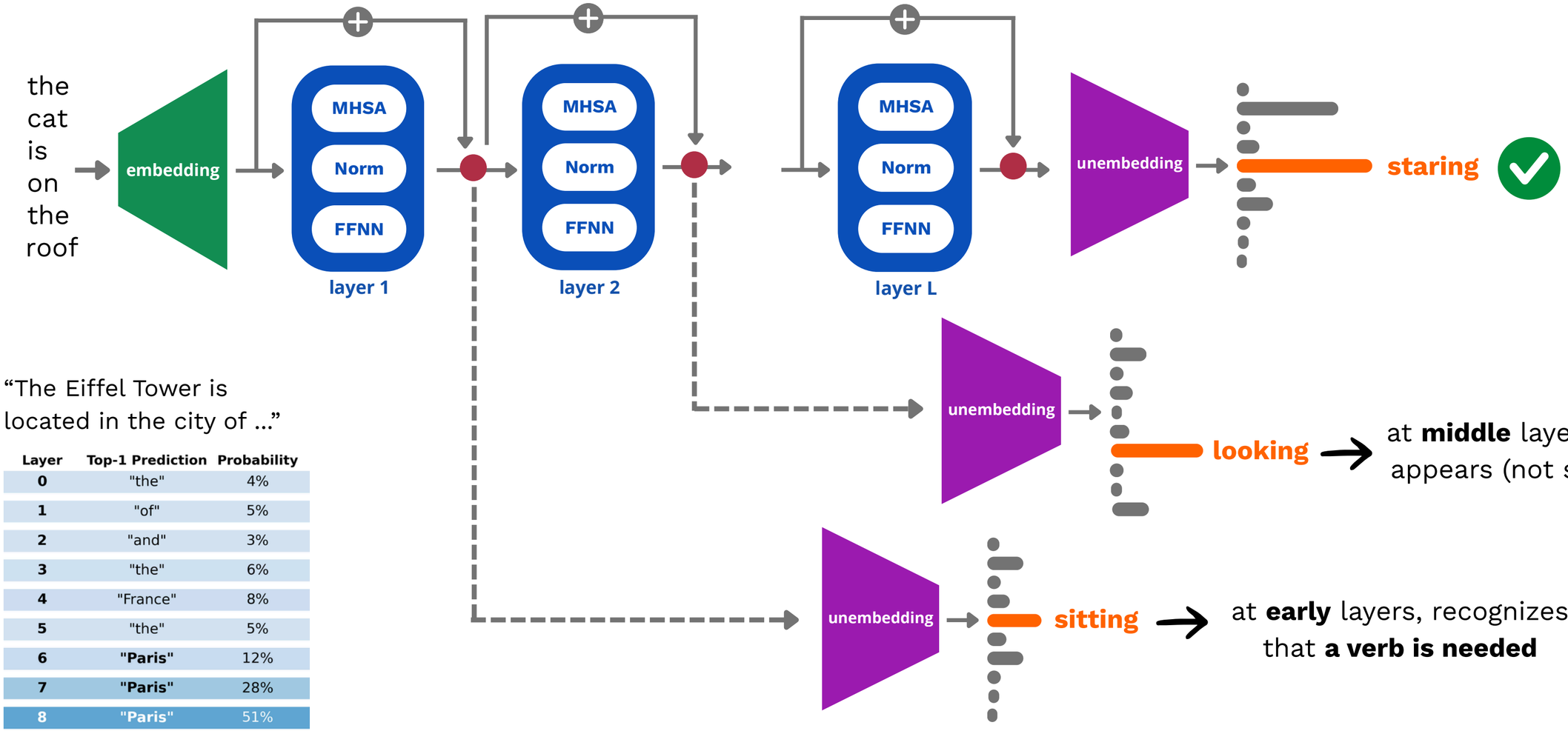
- **positional** rotational **embeddings**;
- a LayerNorm **before the unembedding** matrix;
- different **normalisation** strategies;
- what are **gated MLPs (SwiGLU)**;
- what is a **sampling strategy**;
- fancy choices of **not-vanilla architectures**!

Stop and stare

logit lens and its (recent) evolution, j-lens

Logit Lens

Pretend the model **stopped at layer ℓ**: what would it predict?



"The Eiffel Tower is located in the city of ..."

Layer	Top-1 Prediction	Probability
0	"the"	4%
1	"of"	5%
2	"and"	3%
3	"the"	6%
4	"France"	8%
5	"the"	5%
6	**"Paris"**	12%
7	**"Paris"**	28%
8	**"Paris"**	51%
9	**"Paris"**	72%
10	**"Paris"**	85%
11	**"Paris"**	93%

Logit problems

maybe representations are **not mature enough** to be **unembedded**

Logit Lens (theirs)

	new	simple	network	architecture	,	the	Trans	former
output	model	recurrent	architecture	that	called	attention	former	,
30	model	yet	that	that	called	attention	former	,
27	model	model	that	that	called	**	-	,
24	model	model	called	called	called	**	-	,
21	model	model	that	that	called	âĢ¦"	âĢ¦"	,
18	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
15	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
12	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
9	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
6	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
3	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"	âĢ¦"
input	"	"	"	"	"	"	"	"

Tuned Lens (ours)

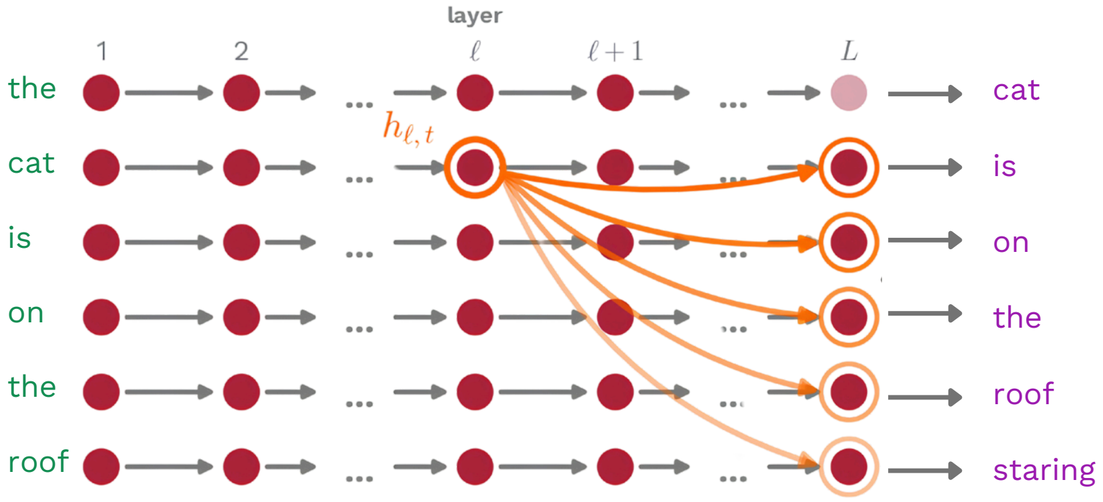
	new	simple	network	architecture	,	the	Trans	former
model	recurrent	architecture	that	called	attention	former	,
model	model	architecture	that	called	attention	former	,
model	model	that	that	called	**	-	,
model	model	that	that	called	**	-	,
model	model	that	that	the	âĢ	-	,
model	model	that	that	the	so	-	,
method	method	that	that	which	so	-	,
method	-	that	that	which	so	-	,
and	and	that	for	which	result	-	.
a	to			and			
,	x	to	,	and	same	_	I
	"	"	"		-	"	"

Tuned Lens (Belrose et al., 2023) learns a small **translator** (an *affine transformation*) **mapping** the residual stream at layer **ℓ** to the **final representation space**.

👍 **interpretable readouts** also at **early** layers
👎 the translator is **trained to match the final output prediction**
👎 **could add some computation not yet present** in the residual stream to do that

J-Lens

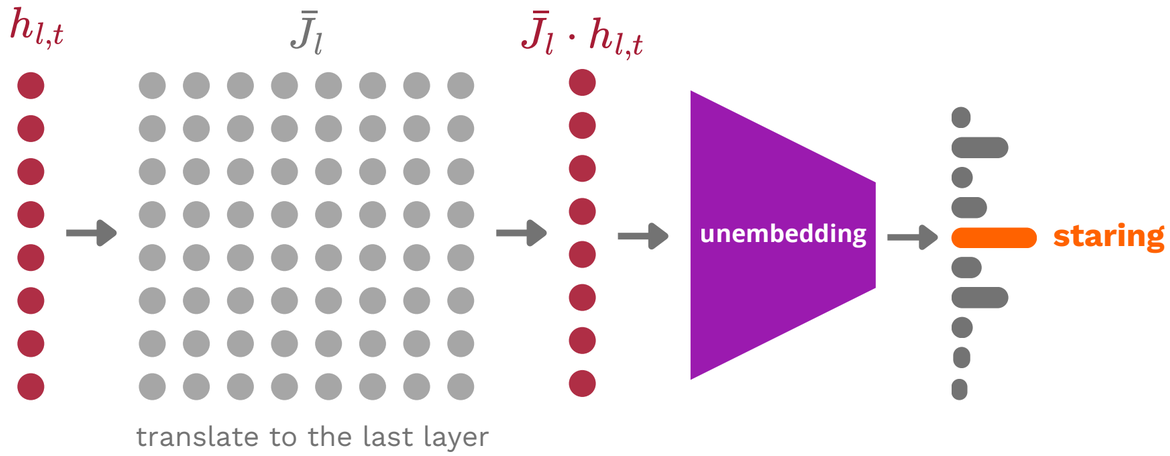
hypothesis: at **any layer**, the rest of the model is **well approximated** by a **single linear map**, ***its first-order Taylor expansion averaged over contexts***



the **hidden state for "cat" at layer ℓ** has a **causal influence** on both **later layers** of "cat" and **every token after** "cat".

we can map this influence with a **Jacobian**

$$J_{l\to L,\, t\to t'} = \frac{\partial h_{L,\,t'}}{\partial h_{l,\,t}} \in \mathbb{R}^{d\times d},\ t' \geq t$$ (each orange arrow)



we **average** across **contexts** and **token positions**, retaining **one Jacobian per layer**:

$$\bar{J}_l = \mathrm{mean}_{t,t'}\ J_{l\to L,\, t\to t'} \in \mathbb{R}^{d\times d}$$

read the **logits** of the **transformed activation**

$$\mathrm{logits} = W_U \cdot \bar{J}_l \cdot h_l,\ W_U \in \mathbb{R}^{|V|\times d},\ h_l \in \mathbb{R}^d$$

Localizing behavior

input attribution and model component importance

Localizing the sources for a behavior

localizing which elements in the **forward pass** are **responsible** for a **specific prediction**

INPUT ATTRIBUTION
estimate the contribution of *input elements* (i.e. **tokens**)

MODEL COMPONENT IMPORTANCE
how various *components* (**representations** and **computational units**) contribute to the model's capabilities

But how strong is the evidence? With the **"ladder of causation"**:

LEVEL 3: COUNTERFACTUAL $y_{a \to \tilde{a}}(x)$

On this input x, had a been different, what would y have been?

this reads as: we found a claim about a certain input x that holds for any a

LEVEL 2: INTERVENTION $p(y \mid \mathrm{do}(a := \tilde{a}))$

If I set a to a chosen value, does y change?

e.g. perturbation, ablation, patching, steering

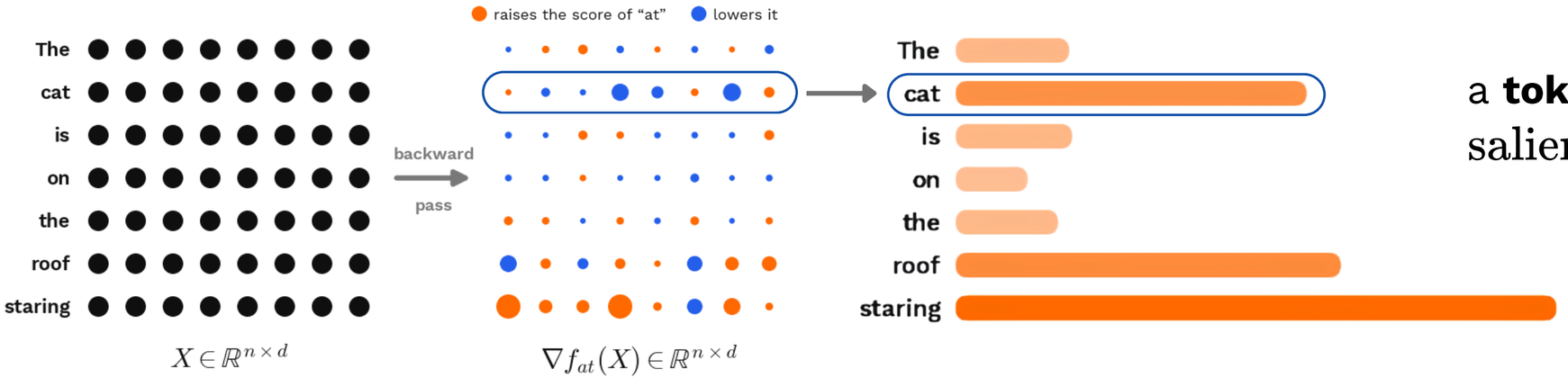
LEVEL 1: ASSOCIATION $p(y \mid a)$

How likely is y, given that I observe a? → e.g. probing, logit lens, attention maps, SAE features; *claiming that something is present* ***DOES NOT*** *mean that it is used by the model/a part of the mechanism*

Gradient input attribution

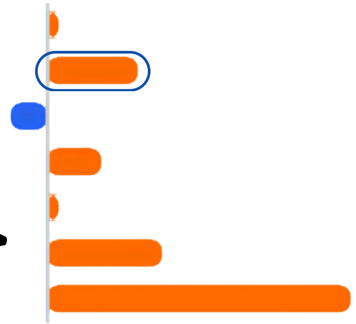
Score every input token by how **sensitive** the **target** word is to it

Given that $f_{at}(X) = \text{logit of } at,\ X = [\mathbf{x}_{The}, \ldots, \mathbf{x}_{staring}] \in \mathbb{R}^{n \times d}$
we can measure **how sensitive** is “at” to each input token by considering its gradient: $\nabla f_{at}(\mathbf{x}) \in \mathbb{R}^{n \times d}$



a **token**-level **aggregation**:
$\text{saliency}_{\mathbf{x}_i} = ||\nabla_{\mathbf{x}_i} f_{at}(\mathbf{x})||_p$

sensitivity is **not** contribution: saliency has no sign and **does not tell in which direction** each input token pushes the prediction



weight by what the token **actually is**:
$\text{Gradient} \times \text{Input}_{\mathbf{x}_i} = \mathbf{x}_i \cdot \nabla_{\mathbf{x}_i} f_{at}(\mathbf{x})$
Gradient×Input = speed **now** × time

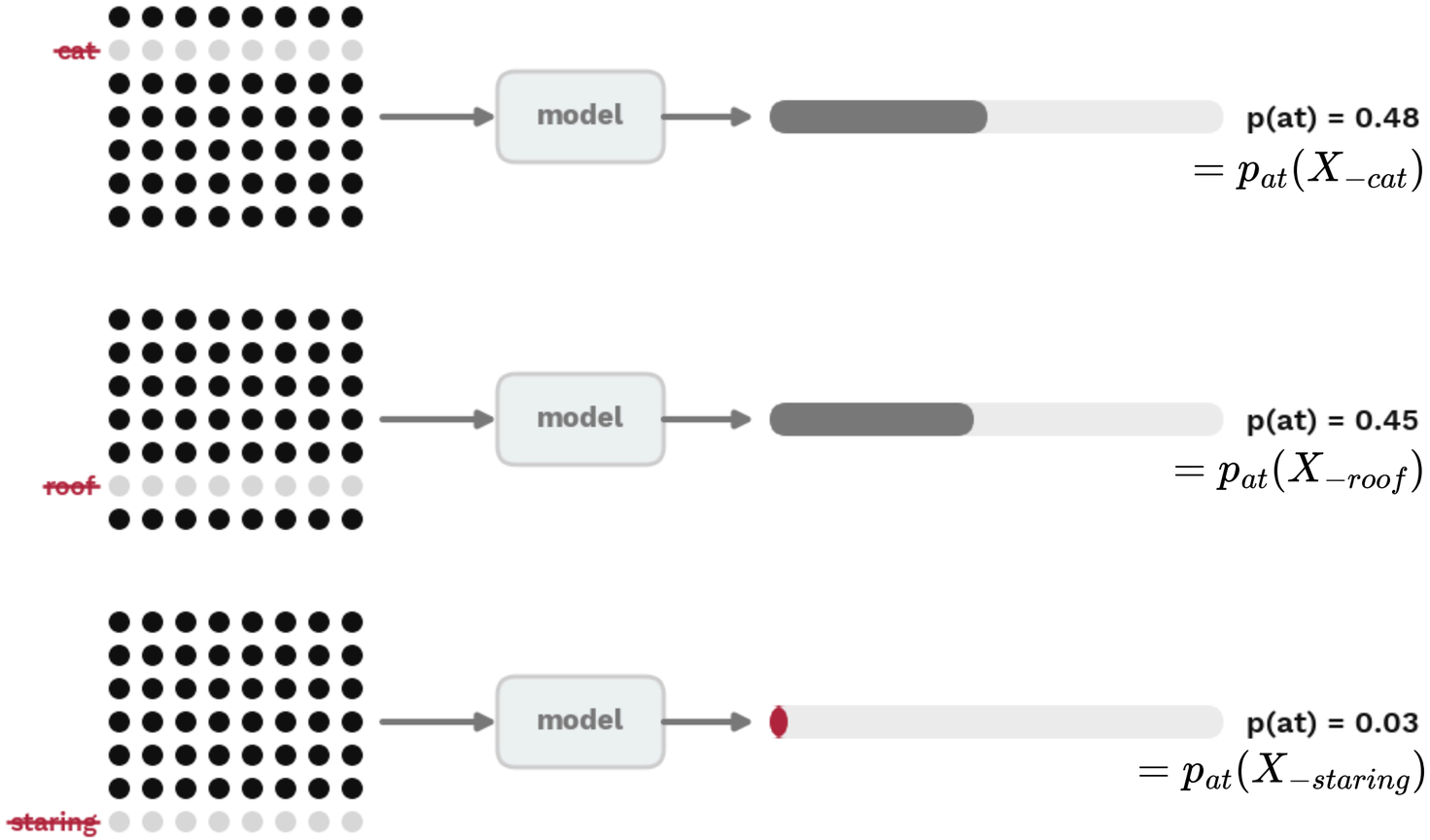
Integrated Gradients = **average** speed × time

$$\text{Integrated Gradients}_{\mathbf{x}_i} = (\mathbf{x}_i - \mathbf{x}_i') \cdot \int_0^1 \nabla \mathbf{x}_i\, f_{at} \cdot (X' + \alpha(X - X'))\, d\alpha$$

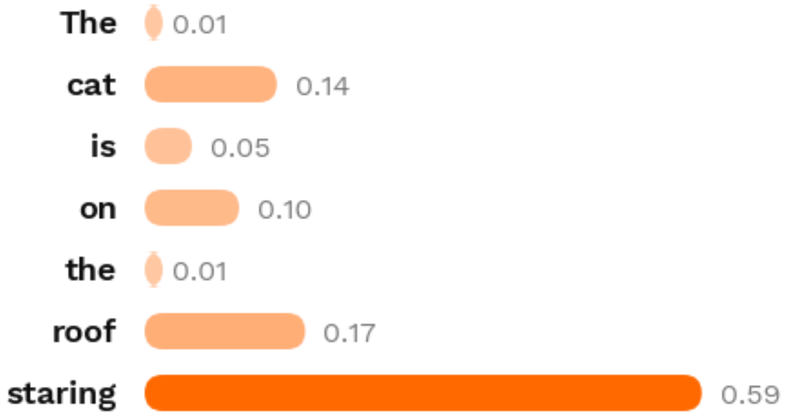
Perturbation-based input attribution

Forget the math, **just remove or modify things** (and hope for the best)

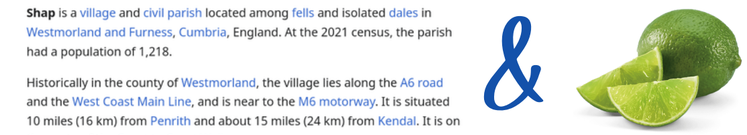
We don't ask how the relevance score *would* move (via gradients), we **directly intervene** on the input and **run a new forward pass**, e.g. by **occlusion:** $\text{occlusion}_{\mathbf{x}_i} = p_{at}(X) - p_{at}(X_{-i})$



If p(at)=**0.62** with the **original** input,



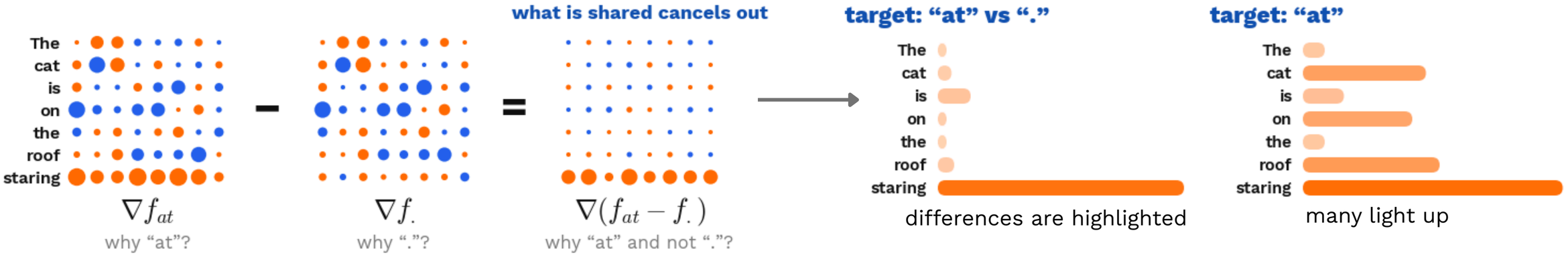
other approaches are based on



Contrastive input attribution

why the model produces a **certain** prediction and **not something else**?

Why "at"? is a vague question: many tokens make a spectrum of continuations plausible!
Pick a ***contrastive option*** and **explain the difference** between the two scores.



We can **export** what we have learnt before **by substituting the *raw logit* with the *logit difference***:

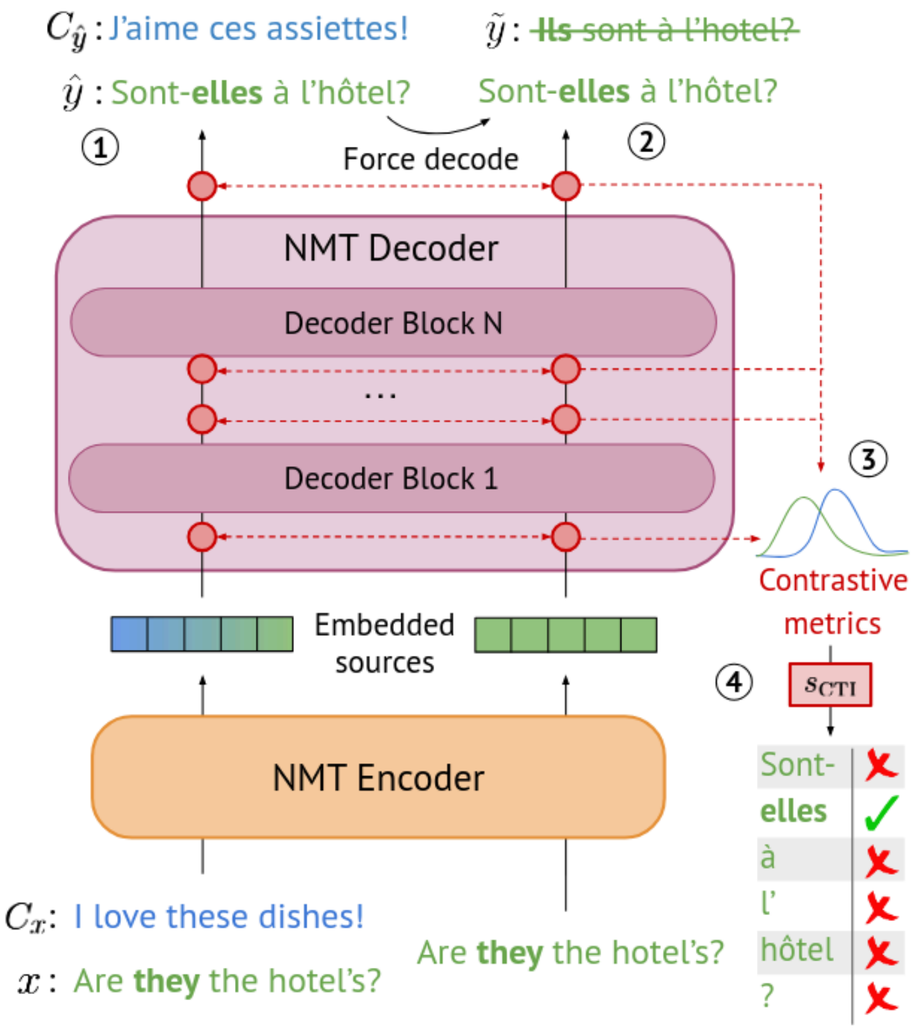
$$f_{at}(X) \rightarrow f_{at}(X) - f_{contrastive}(X)$$

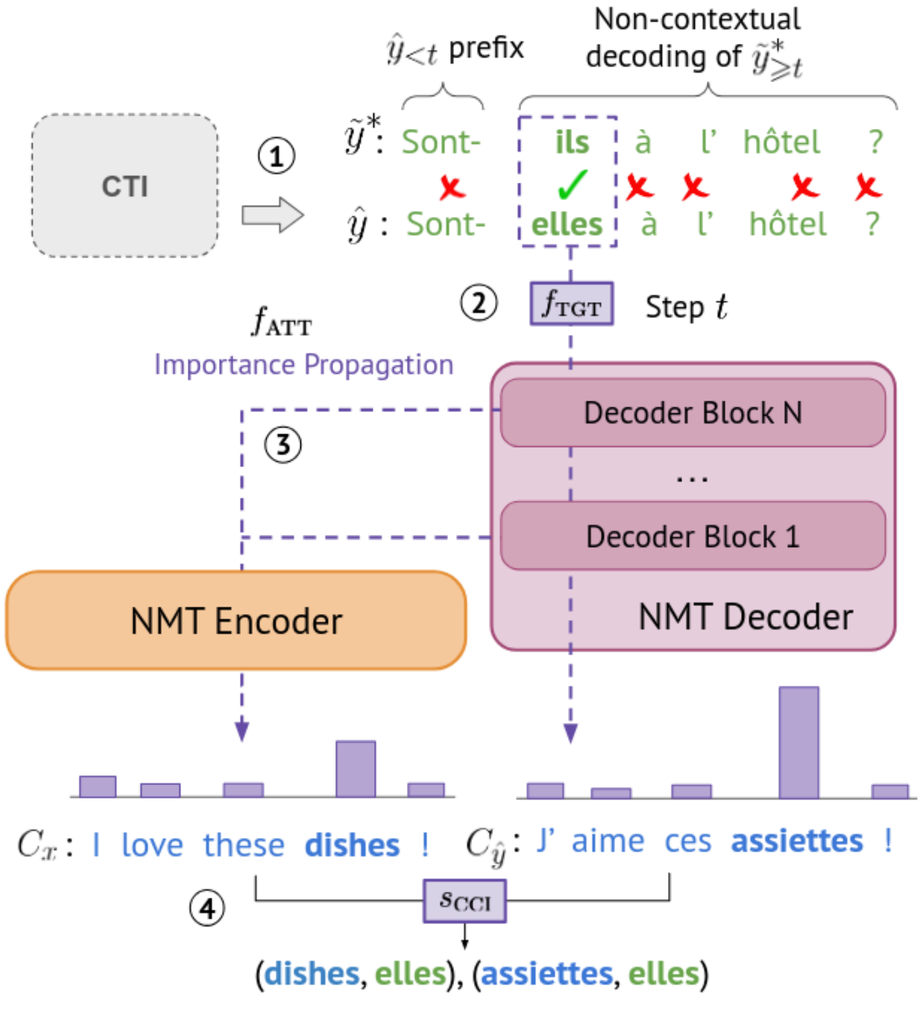
- contrastive **saliency:** $||\nabla_{\mathrm{x}_i}\,(f_{at} - f_{contrastive})(X)||_p$
- contrastive **Gradient x Input:** $\mathrm{x}_i \cdot \nabla_{\mathrm{x}_i}\,(f_{at} - f_{contrastive})(X)$
- contrastive **occlusion:** $[f_{at} - f_{contrastive}](X) - [f_{at} - f_{contrastive}](X_{-i})$

How should we **choose** the **contrastive examples**?

Is it actionable and actual?

1. **contrastively** identify **context-sensitive target tokens** in generated texts
2. **link** them to **contextual cues** justifying their prediction.

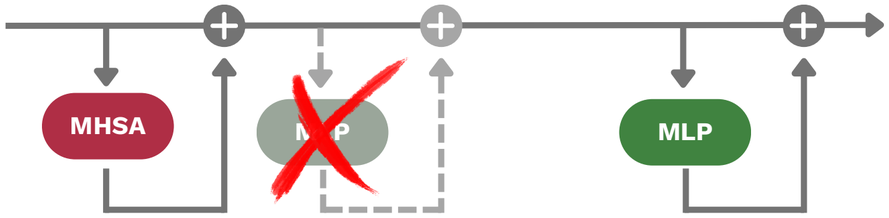




Model component importance

switch from tokens to **computational units**

ABLATION: switch off one component and see how much “at” (its logit) **suffers**

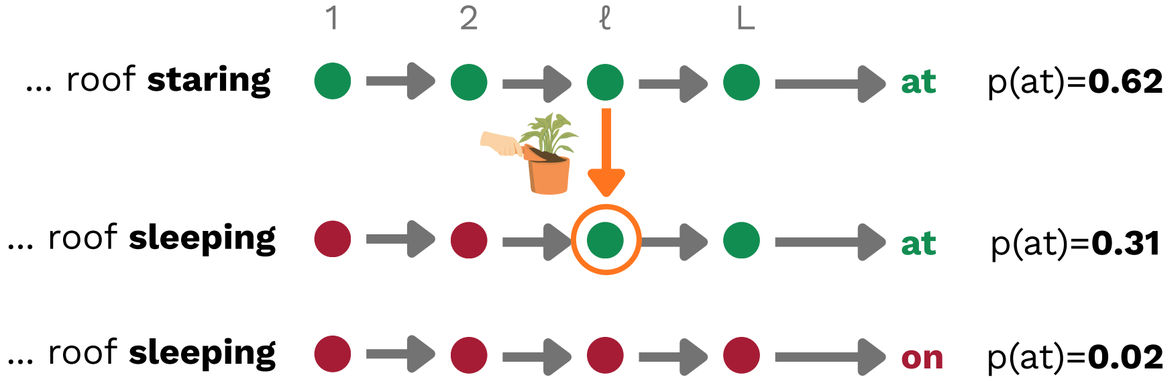


replace the output of a component with another value:

e.g. zero ($\bar{a} = 0$), mean ($\bar{a} = \mathbb{E}[a]$), ...

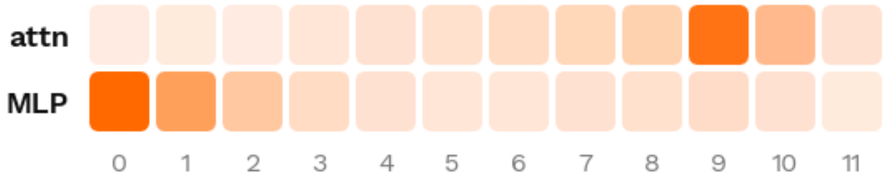
ACTIVATION PATCHING: replace what a component **writes** with ***its value from another input***

Two runs: one clean (“**staring**”), one corrupted (“**sleeping**”). **Inject** the state of the clean run into the corrupted run. Like an alchemist, see what happens



$$\text{indirect effect} = f_{at}(X_{patched}) - f_{at}(X_{corrupted})$$
$$\text{total effect} = f_{at}(X_{clean}) - f_{at}(X_{corrupted})$$

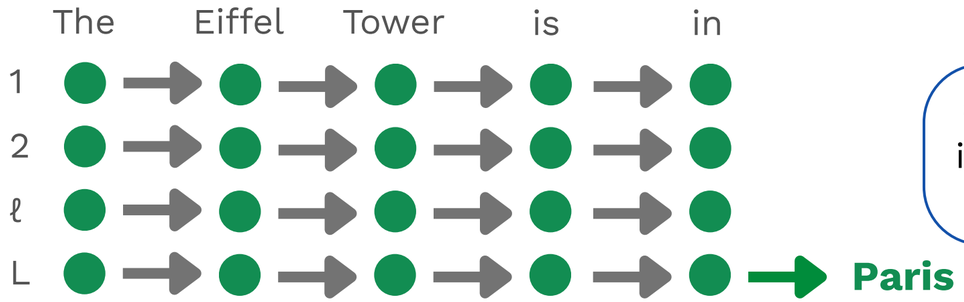
$$\text{restored effect} = \text{indirect}/\text{total}$$



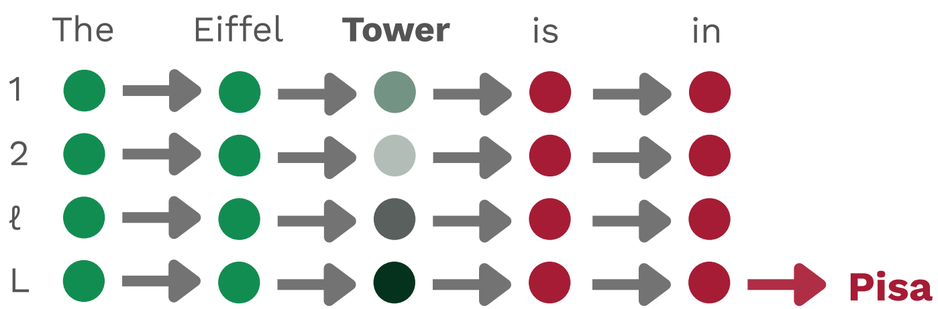
Causal tracing

use activation patching to **discover** where the model **recalls a fact**

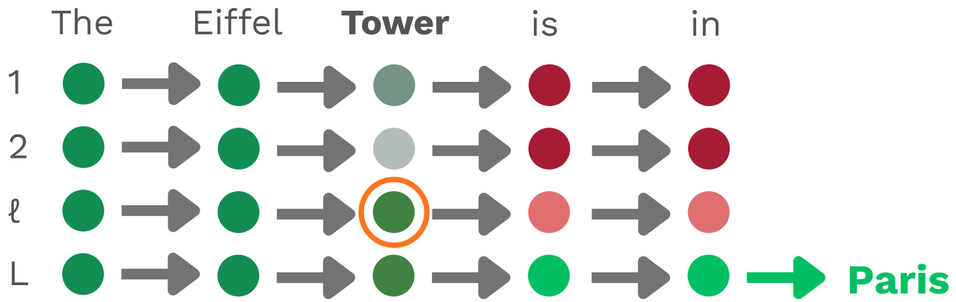
Somewhere in its weights “Eiffel Tower” → “Paris”. **At which layer and on which token?**



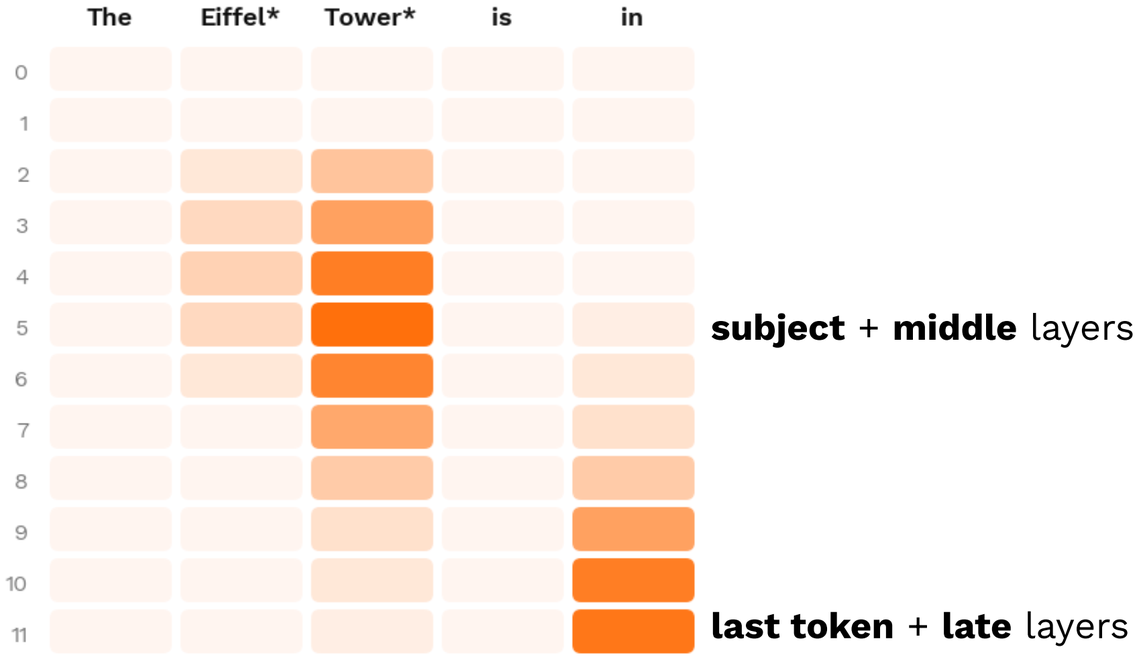
add Gaussian noise:
it does not recognise
Paris anymore



→ **corrupting “Tower”** causes a wrong prediction.
Restore the correct state at a certain layer:



despite the corruption, repotting “Tower”
in layer ℓ **is sufficient to recover** “Paris”

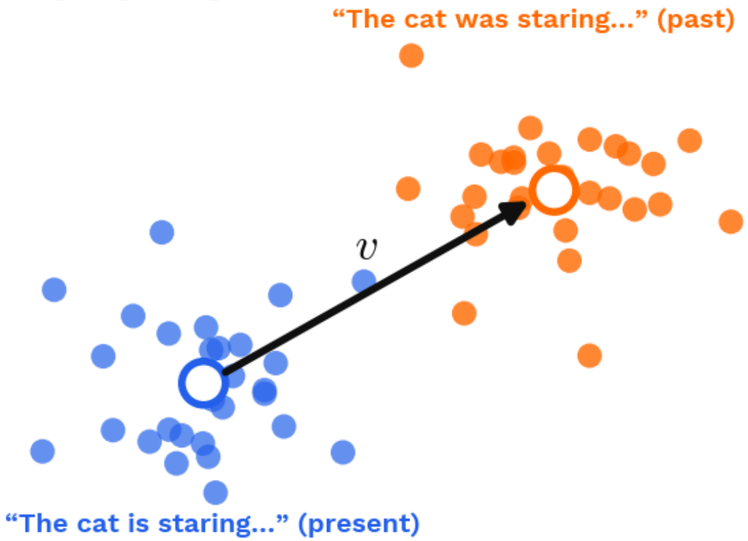


Representations and concepts

LRH, steering and function vectors, probes, SAEs

Linear Representation Hypothesis

hypothesise that **concepts** are **directions** in the activation space



If true, sentences **with**/**without** the concept **should sit at the opposite sides** of the activation space

Q1: How do we **find** that direction?
By the **difference of the cluster centroids**:

$$\mathrm{DiM}(c) = \mathrm{mean}(h_l \mid \mathrm{with}_c) - \mathrm{mean}(h_l \mid \mathrm{without}_c) = \vec{v}_c$$

Q2: How can we **induce** that concept into **concept-depurated examples**?
By adding a **steering vector** to the examples' activations:

$$h_l^{steered} = h_l + \alpha \cdot \vec{v}_c, \ \alpha = \text{steering strength}$$



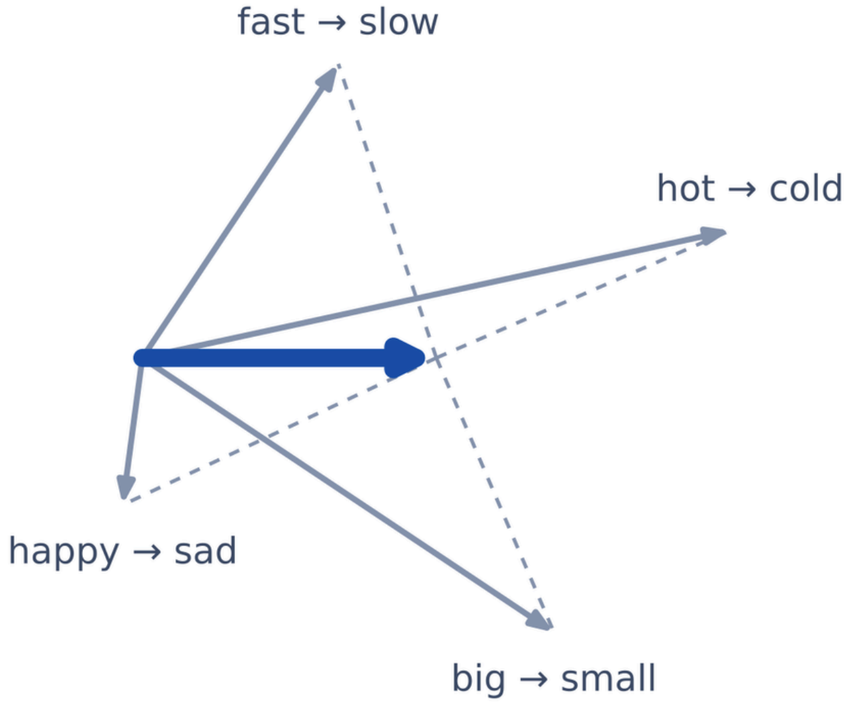
why do we need the steering strength?

how can we choose the **right value** for it?

are tracking down the **right concept**?

Function vectors

it is possible to **condense entire tasks** into a single vector



TASK: *find the opposite*
EXAMPLES: hot → cold, big → small, happy → sad, fast → slow

1. take the hidden state at the **last token** at a certain layer
2. average it across prompts → **task vector**
   (tbf the original **function vector** considered only **MHSA** outputs)
3. **patch** it into an unseen example at the same layer: light → dark

We refer to this phenomenon as **In-Context Learning → in-context vectors**

STEERING VECTORS

human-engineered concept directions that **modify behavior**

FUNCTION VECTORS

naturally occurring directions that **encode tasks**, **learnt** by the model **autonomously**

it has been shown that function vectors exhibit **compositionality**

Is something here? Probe it

break free from hand-crafted directions and **let a classifier** find them

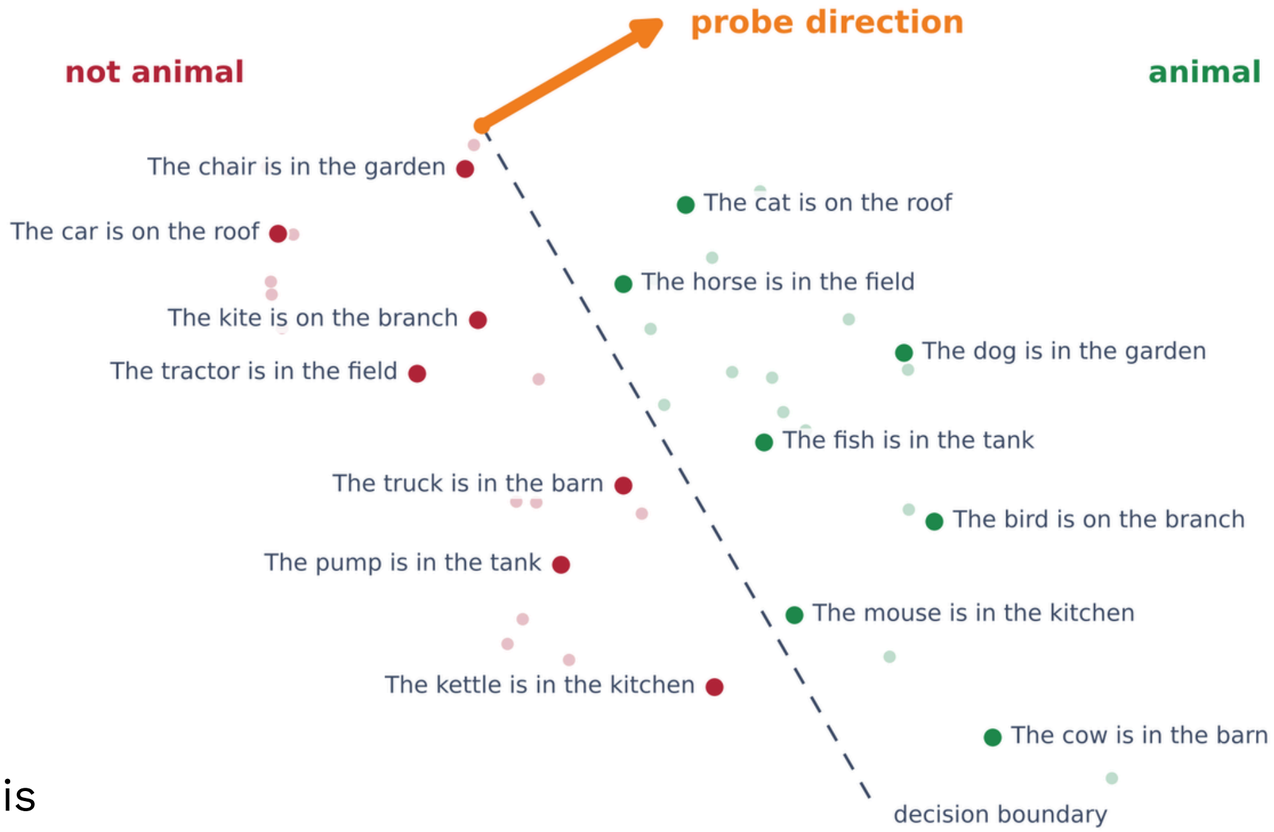
We only provide **labels**, a classifier **learns** the direction:

1. collect **hidden states** at a layer, each with a **label**;
2. **train a linear classifier** (a *logistic regression*) to predict the label;
3. **its weight vector is the direction** encoding that property.

The probe is a **small linear classifier** on top of a **frozen model**:

$$\hat{y} = \sigma(w \cdot h_l + b)$$

Remember that **decodable ≠ used by the model**. **Steering** the model along the identified direction is the **causal evidence** needed.



Superposition

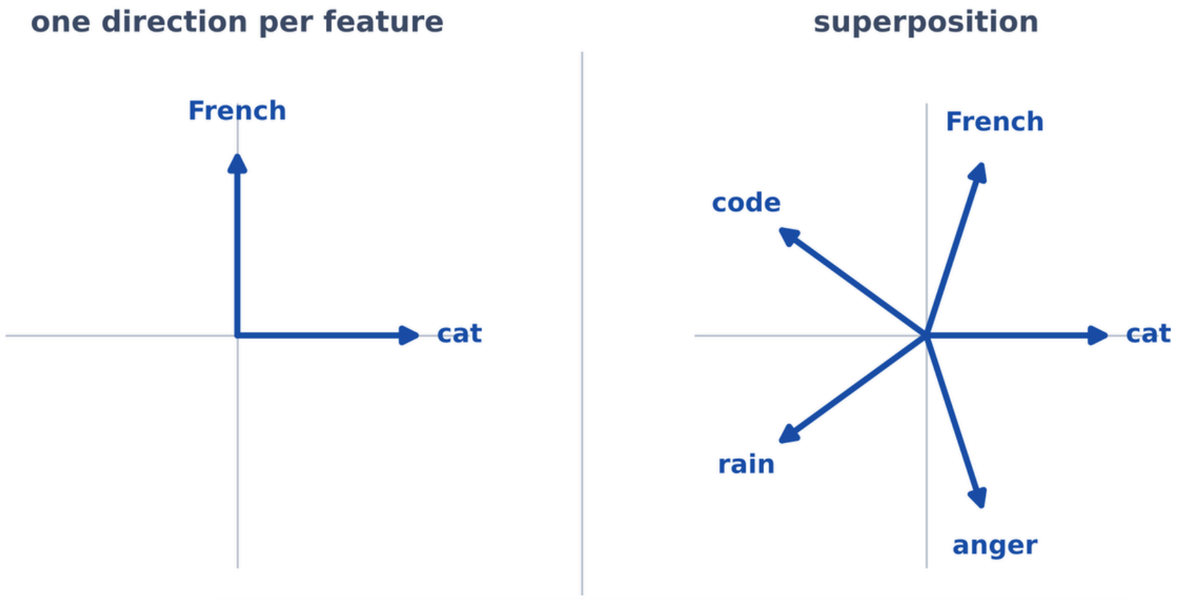
d dimensions but **need to represent millions** of features, how?

*If every **concept** had its **own direction*** (exactly like a one-hot encoding), a d-dimensional stream could hold **only d of them**

Superposition is the phenomenon with which the model stores **features as quasi-orthogonal directions**, *accepting some interference between features that rarely co-occur*.

CONSEQUENCES?

1. individual neurons are ***polysemantic***, i.e. *respond to multiple unrelated features*
   → **extracting concepts** from individual neurons is **non-trivial**
2. the model has probably **compressed representations** through a **certain *geometry***
   → this is why we search for **directions encoding concepts** (steering & function vectors, probing)

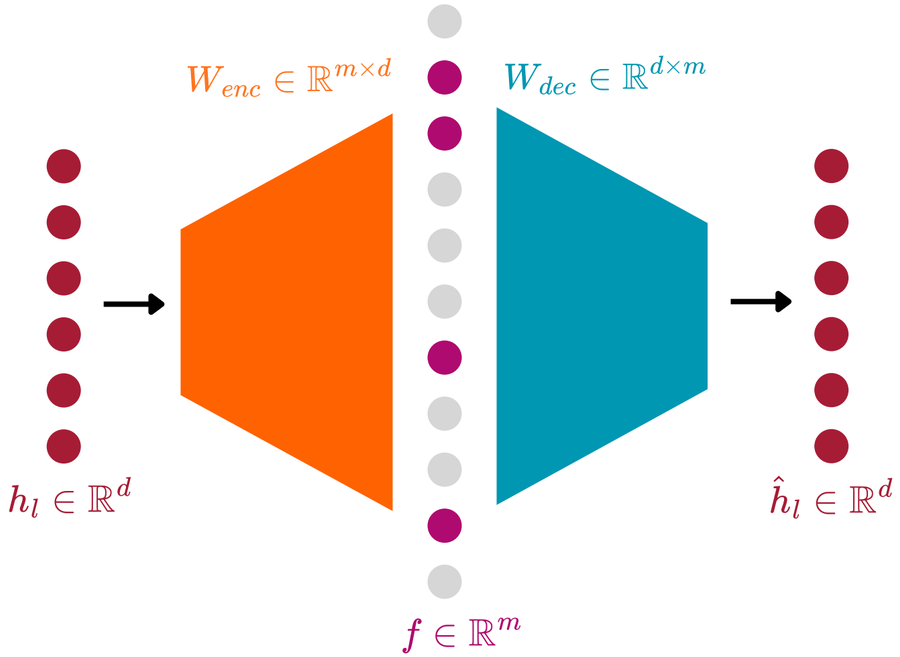


what happens to *cosine similarities* here?

Sparse AutoEncoders (SAEs)

doing the non-trivial task of **disentangling concepts from neurons**

1. collect **hidden states/activations** at a layer;
2. **a small NN** expands them to **m >> d** dimensions;
3. train this NN to **rebuild the original** activations.



train with a **sparsity penalty**, forcing the latent state to use **just a few features per example:**

$$L = ||h_l - \hat{h}_l||^2 + \lambda \cdot ||f||_1$$

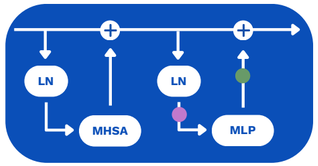
where $f = \text{ReLU}(W_{enc} \cdot h_l + b)$, and $\hat{h}_l = W_{dec} \cdot f$

SAEs learn **a dictionary of m almost-orthogonal directions** in the original d-dimensional activation space

→ each **column** of the **decoding** matrix is a learnt feature direction

👎 again, **reconstructed ≠ used** by the model
👎 strongly **dependent** on the **regularization choices**
👎 some features remain **unexplained** (*dark matter*)
👎 **unnamed** SAE features → eyeballing and extra biases injected

Transcoders

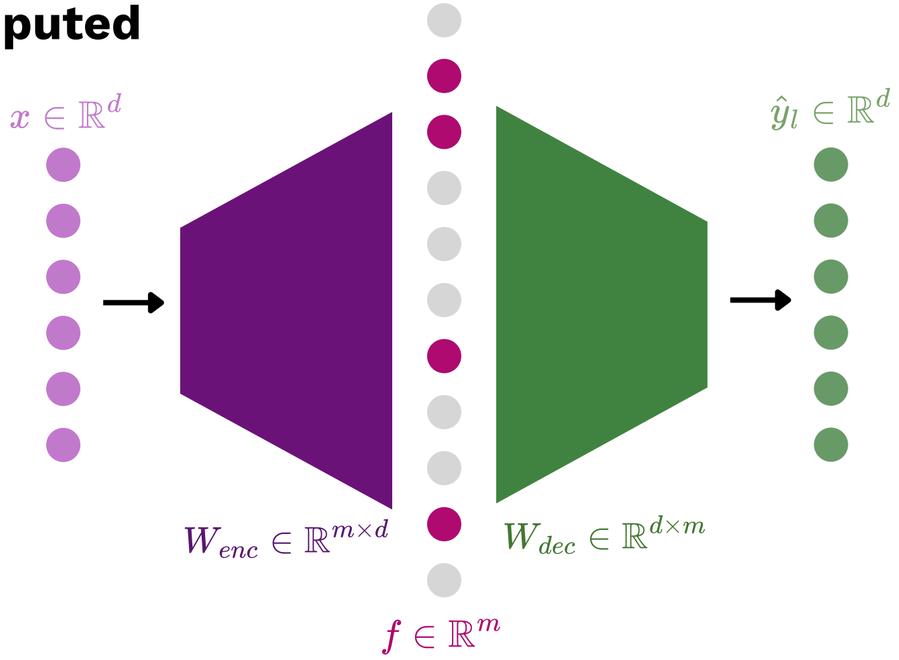


SAEs show what is **represented**, transcoders show **how** it is **computed**

1. collect the **input** of a **MLP** at a layer;
2. **expand** it to m >> d **sparse features** (as a SAE);
3. train it to **predict the MLP's output**.

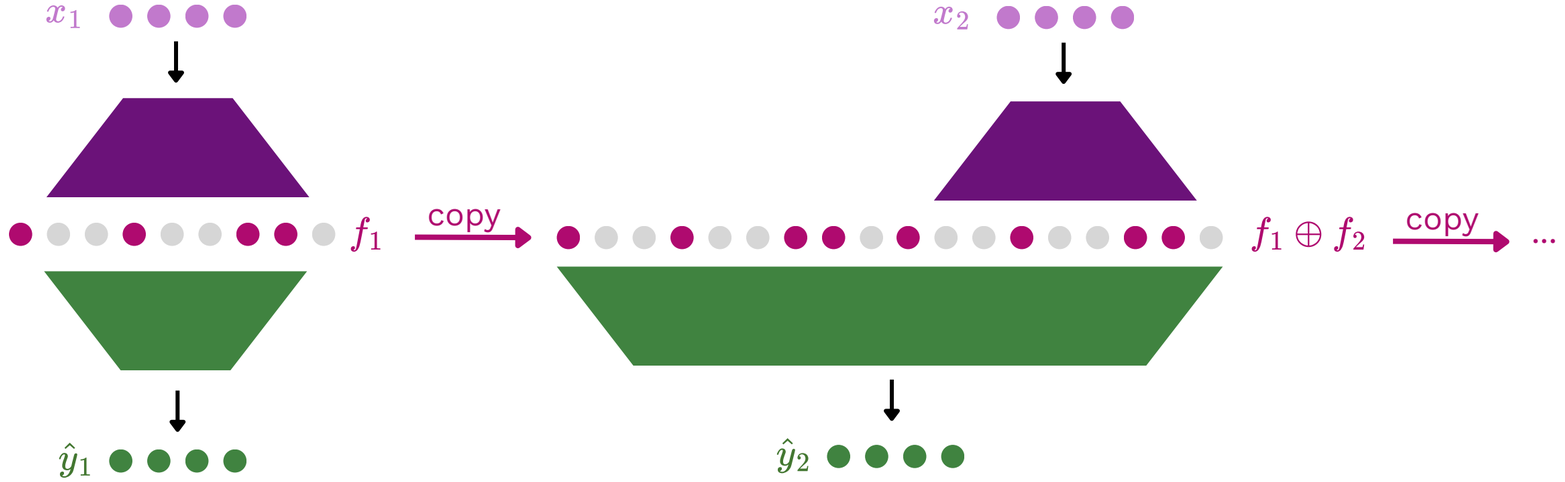
TL;DR: replace the MLP with a *sparse version* of itself

$$L = ||\mathrm{MLP}_l(x) - \hat{y}_l||^2 + \lambda \cdot ||f||_1$$



Can we replace **every MLP** with a **more interpretable counterpart?**

Cross-Layer Transcoders:



- one *encoder* per layer: **features** depend on the **current input layer** only
- layer-incremental *decoders*: **information** may be needed **later**, needs to be carried over